Assignments for the Fitoptivis Workshop

Freek van den Berg, Hossein Elahi, Shayan Tabatabaei Nikkhah

September 16, 2019

1 Introduction

A Cyber-Physical System (CPS) comprises the integration of computation, software, networking, and physical processes. An imaging system is a CPS that processes images and for which safety and performance are key features.

To achieve a right balance between safety and performance of imaging systems, the FitOptiVis project includes the development of an integral approach for smart integration of image-processing pipelines architecture, as well as supported tools for design-time as well as run-time.

For this purpose, a Domain Specific Language (DSL) and supporting toolset has been developed, also referred to as the FitOptiVis component abstraction model. The DSL provides a language in which systems are specified in an integral way, whereas the toolset enables the automated optimisation within systems.

In this workshop, you will be introduced to the FitOptiVis component abstraction model using three assignments (cf. Section 3 till 5, respectively). The assignments will both focus on the DSL and the toolset. The first assignment will teach you how to open and run a given example in this model. Next, the second assignment focuses on adding a new configuration as well as a new component to an existing system. After this, the third assignment emphasizes the creation a compound component. Finally, we will ask you to model your own system of choice. Each assignment will contain learning goals (indicated by $\mathcal{L}$) which are achieved by performing a number of actions (indicated by $\mathcal{A}$).

We conclude with a **short survey** in the end, which assesses how you have experienced this workshop. The results of this survey are used to improve workshops that might be held in the future and to increase the quality of the DSL and belonging toolset.

2 Assignment 0: Preliminaries

In this preliminary assignment, we will install and run the aforementioned DSL and toolset. Preferably, please perform the following actions before the workshop.

$\mathcal{A}$2.1 Download **VirtualBox** for your operating system at `http://www.virtualbox.org/wiki/downloads` if you do not have **VirtualBox** installed.

$\mathcal{A}$2.2 Download the DSL and toolset from `http://www.es.ele.tue.nl/~20180469/FitoptivisWorkshop11Sep2019.zip` and store it in an empty folder on your computer.

$\mathcal{A}$2.3 Open **FitOptiVis.ova** from the folder (as selected in Action *$\mathcal{A}$2.1*) by double clicking on it.

$\mathcal{A}$2.4 Press import (make sure you have at least 15 GBs free space on the drive on which you want to import). After some time, you should see the FitOptiVis button appear on the left side as Figure 1 conveys

Figure 1: Imported virtual OS

$\mathcal{A}$2.5 By double clicking this FitOptivis button, the desktop of the virtual machine appears on which you are going to perform the assignments of this workshop. If you get an error regarding the so-called **VT-x** being disabled, you have to enable the so-called **VT-x** and **VT-d** in the BIOS. The assistants of this workshop are glad to help you with this, but you can also fix the problem yourself, as follows. Restart your computer and press the BIOS key (e.g., **F10**) to make the BIOS menu appear. Then look for virtualization settings in the menu (e.g., under Advanced tab) and enable the **VTx** and **VTd**. Save your changes and exit.

$\mathcal{A}$2.6 To adjust the resolution, press **Right ctrl+F** to exit the full-screen mode, and then right click on the screen icon shown in Figure 2. Select **Vir-**

tual Screen 1, and choose a proper resolution and/or scaling percentage. Pressing **Right ctrl+F** again brings you back to the full-screen mode.

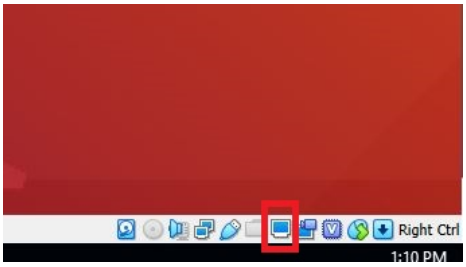

Figure 2: Desktop virtual OS

A2.7 Open **Eclipse** by double clicking on **Eclipse** icon on the desktop of the virtual machine (as in Figure 3). A windows titled "workspace_workshop - Eclipse IDE" appears which comprises five projects on the left.

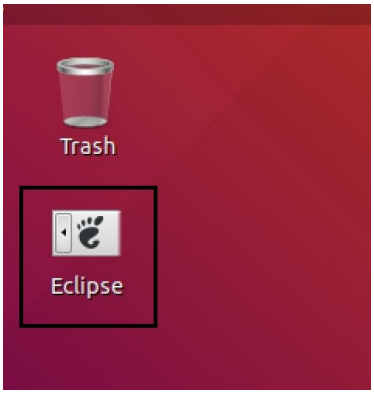

Figure 3: Virtual OS desktop

3 Assignment 1: Generate visualizations

In this assignment, you will become familiar with the FitOptiVis DSL component abstraction model. For this purpose, Figure 4 depicts a model of a smart camera (see also FitOptiVis deliverable D2.1) which is a hardware component that is used to enable Bio-metric Access Control task.

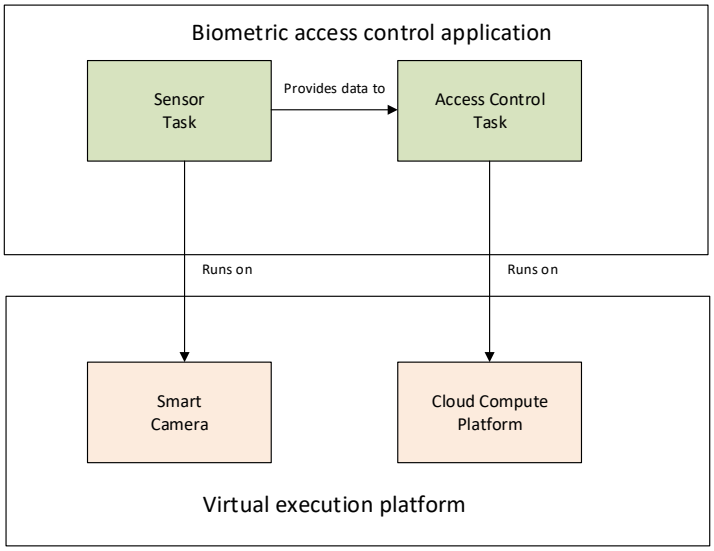

Figure 4: Example component model architecture of a bio-metric access application

This assignment conveys the following two learning goals.

$\mathcal{L}$*3.1* Running a FitOptVis DSL example

$\mathcal{L}$*3.2* Examining the results which include visualizations

To accomplish these learning goals, you are expected to perform the following actions.

$\mathcal{A}$*3.1* We assume that you have a virtual machine instance open (as accomplished in action $\mathcal{A}$*2.5*). In this virtual machine, open **Eclipse** by double clicking on the corresponding icon as described in action $\mathcal{A}$*2.7*.

$\mathcal{A}$*3.2* Next, run the so-called **FitOptVis DSL** by clicking on the triangular on the right side of the green play button (as displayed in Figure 5) and select **Launch Runtime Eclipse** from the pull-down menu. After quite a while the second Eclipse will start (see Figure 6). In this manual, we refer to this Eclipse as the Runtime Eclipse. We appreciate your patience.

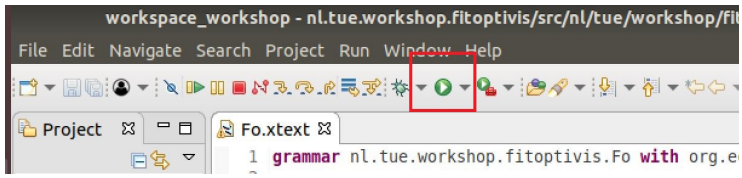

Figure 5: Run DSL

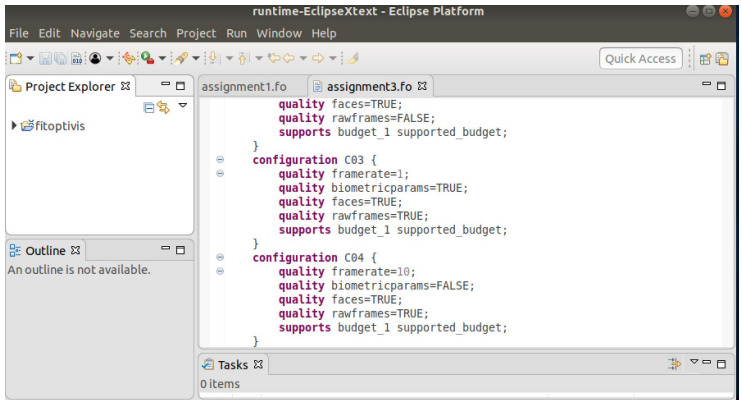

Figure 6: Import virtual OS

A3.3 In the Runtime **Eclipse**, open the so-called **DSL instance** with name **assignment1.fo**. It can be found in main folder of the project (as shown Figure 7). A DSL instance specifies a system that consists of components. The first component specified is the top-level component which generally refers to the other components. We invite you to have closer look at the system that implements what is depicted in Figure 4.

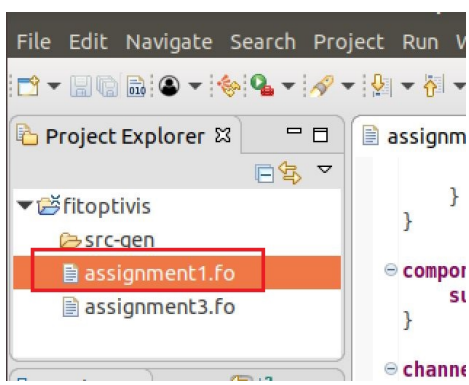

Figure 7: Open a given model

A3.4 Make a negligible change, for example, add a space. Save the instance by pressing **Ctrl+s**

A3.5 Build the instance by pressing **Ctrl+b** or by clicking on **Build All** under **Project** tab, and wait until the building is completed. Bear in mind that when you make changes to multiple files and save them, **Build All** yields the building of multiple files, which is usually very slow. We therefore recommend you to only work on one file at a time.

A3.6 After the instance has been built, the results are stored in a web-page in the **src-gen** folder that can be accessed from the project explorer (see also Figure 7). After double clicking on **src-gen**, double click the folder that starts with the name of your DSL instance and further contains the exact time it was build (i.e., year-month-day hour minute second milli-second). Finally, open **results.html** (see also Figure 8). You should now see a table of contents containing hyperlinks to results.

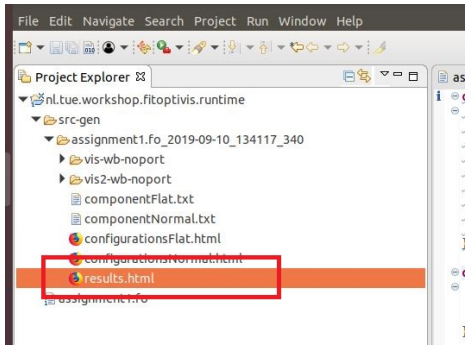

Figure 8: Open the results

A3.7 The file results.html displays the following results:

- The original Fitoptivis specification: the original DSL instance
- A textual tree representation of the top component: a textual representation of the top component including its subcomponents
- An overview of the possible configurations: all configurations possible
- Hierarchical visualization and parameters of the components: the hierarchy of components and their parameters
- The budget connections of the components: the way components depend on each other via budgets
- The I/O connections of the components: the way components depend on each other via I/o

Note that hyperlinks on top of the page facilitate browsing through the different results. Additionally, there are links to go back to the top of the page again too.

4 Assignment 2: Specify a component

In this assignment, a new configuration is added to the smart camera component of the given example. Besides this, a new task and its connections to run on the so-called virtual execution platform are added.

$\mathcal{L}$4.1 Modifying a component by adding a new configuration

$\mathcal{L}$4.2 Defining a new component from scratch

$\mathcal{L}$4.3 Specifying budget, input and output connections of the components

$\mathcal{L}$4.4 Examining the results which include visualizations (in line with *$\mathcal{L}$3.2*)

To achieve these learning goals, we require you to put the following actions into place.

$\mathcal{A}$4.1 Copy the DSL instance **assignment1.fo** by right clicking on it and selecting **Copy**. Paste it in Project Explorer by right clicking at an empty spot in the Project Explorer and choosing **Paste**. Type **assignment2.fo** and press **OK** to create an additional file named **assignment2.fo** for the second assignment.

$\mathcal{A}$4.2 Open DSL instance **assignment2.fo** in the **Project Explorer** by double clicking on it. You should now see the system you have created in assignment 1.

$\mathcal{A}$4.3 Add a new configuration to the "Smart Camera" and name it **C06**, viz., add Mode 6 of the smart camera based on the information in Table 1, right after the configuration **C05**. Hint: the five given configurations in the DSL instance indicate how this is done.

Table 1: Smart camera component configurations

Mode	Frame rate (fps)	Biometric parameters	Faces	Raw frames
1	1	+	-	-
2	1	+	+	-
3	1	+	+	+
4	10	-	+	+
5	10	-	-	+
6	30	-	-	+

$\mathcal{A}$4.4 Add a new task class to the model (see Figure 9) which logs information of the system. For this purpose, the creation of a component class named **LoggingInformationTask** is needed right after the **AccessControlTask** component. Hint: this component class has a syntax that is similar to the one of the syntax of **AccessContorlTask** class.

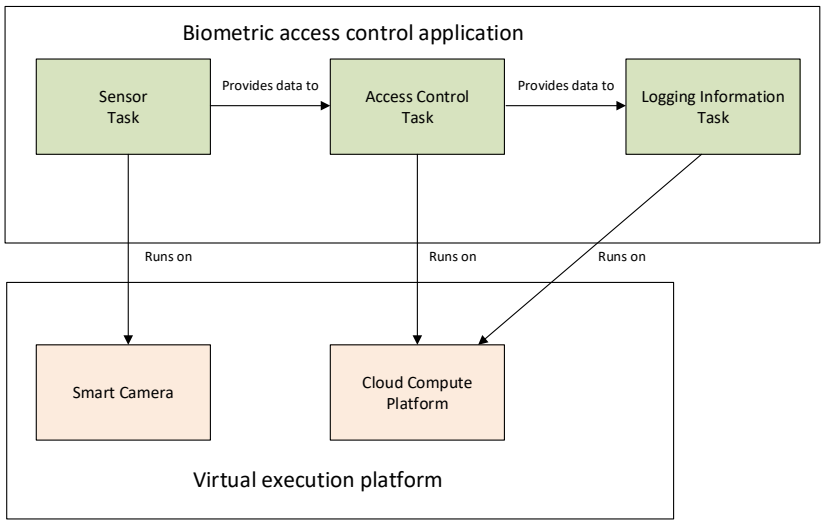

Figure 9: An extension of the smart camera example

$\mathcal{A}$4.5 Next, we model the **LoggingInformationTask** and its connections as depicted Figure 9. We start by defining an instance of this component like **line 4** in Table 2. After this, a budget connection of this component is specified to indicate it runs on **CloudComputePlatform**. That is, a so-called "runs on" statement as can be seen in **line 10** of Table 2 needs to be added. Finally, to capture the data dependency of this component, a so-called "outputs to" statement as in **line 13** of Table 2 needs to be added.

$\mathcal{A}$4.6 To visualize the changes in this assignment, save and build by repeating action *$\mathcal{A}$3.4* and *$\mathcal{A}$3.5*.

$\mathcal{A}$4.7 After the instance has been built, the results are stored in a web-page in the **src-gen** folder that can be accessed from the project explorer (see also Figure 8). After double clicking on **src-gen**, double click the folder that starts with the name of your DSL instance and further contains the exact time it was build (i.e., year-month-day hour minute second milli-second). Finally, open **results.html**. You should now see a table of contents containing hyperlinks to results.

Table 2: Add a component instance and its connections

```
component D21example4dot3 {
    component SensorTask st;
    component AccessControlTask act;
    component LoggingInformationTask lit;
    component SmartCamera sc;
    component CloudComputePlatform ccp;

    required_budget.st runs on supported_budget.sc;
    required_budget.act runs on supported_budget.ccp;
    required_budget.lit runs on supported_budget.ccp;

    out.st outputs to inp.act;
    out.act outputs to inp.lit;
}
```

5 Assignment 3: Define a compound component

This assignment teaches you how to define a compound component which consists of multiple compoments. In this particular case, the compound component is a composition of four components such as depicted in Figure 10. We identify the following three learning goals.

$\mathcal{L}$5.1 Creating a compound component by adding basic components to a component

$\mathcal{L}$5.2 Specifying budget, input and output connections of basic components

$\mathcal{L}$5.3 Examining the results which include visualizations (in line with *$\mathcal{L}$3.2*)

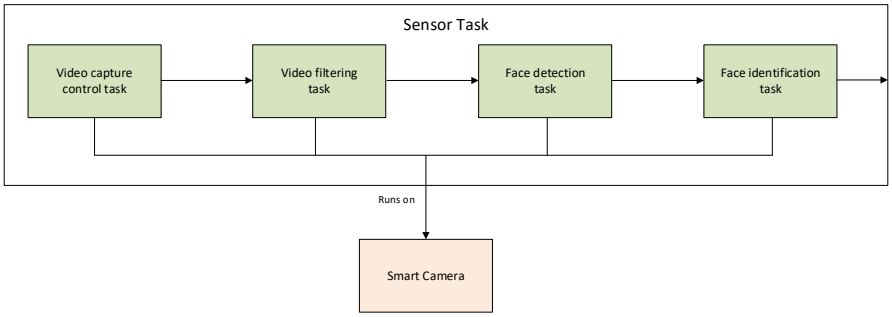

Figure 10: Example of the biometric access application with decomposed sensor tasks

The three learning goals are realized using the following actions.

$\mathcal{A}$5.1 Copy DSL instance **assignment2.fo** and paste it with the name **assignment3.fo** (cf. *$\mathcal{A}$4.1*). Next, open DSL instance **assignment3.fo** by double clicking on it. Once again, you should see the system that you have created in the previous assignment.

$\mathcal{A}$5.2 Figure 10 displays one component in **SensortTask** (in red) which decomposes into four components (in green). Define classes for these four components inside **SensorTask** in a similar fashion as action *$\mathcal{A}$4.4*

$\mathcal{A}$5.3 Add instances of these four basic components into the sensor task component by adding **lines 2-5** of Table 3. You are free to name the instances as you wish, but need to use this exact name when referring to them. Note that Table 2 (lines 2-6) provides another example of how instances are added.

$\mathcal{A}$5.4 Add the respective budget connections of these four components by adding **lines 8-11** of Table 3 to your system.

Table 3: Add four basic components and their connections

```
1.  component SensorTask {
2.      component VideoCaptureControlTask vcct;
3.      component VideoFilteringTask vft;
4.      component FaceRecognitionTask frt;
5.      component FaceIdentificationTask fit;
6.
7.      outputs regular_channel output;
8.      inputs regular_channel output;
9.
10.     required_budget.vcct runs on required_budget;
11.     required_budget.vft runs on required_budget;
12.     required_budget.frt runs on required_budget;
13.     required_budget.fit runs on required_budget;
14.
15.     inp outputs to inp.vcct;
16.     out.vcct outputs to inp.vft;
17.     out.vft outputs to inp.frt;
18.     out.frt outputs to inp.fit;
19.     out.fit outputs to out;
20.     requires regular_budget required_budget;
21. }
```

$\mathcal{A}$5.5 Add the respective I/O connections between the basic components by adding **lines 12-16** of Table 3 to your system.

$\mathcal{A}$5.6 To visualize the changes in this assignment, save and build by repeating *$\mathcal{A}$3.4* and *$\mathcal{A}$3.5*.

$\mathcal{A}$5.7 After the instance has been built, the results are stored in a web-page in the **src-gen** folder that can be accessed from the project explorer (see also Figure 8). After double clicking on **src-gen**, double click the folder that starts with the name of your DSL instance and further contains the exact time it was build (i.e., year-month-day hour minute second milli-second). Finally, open **results.html**. You should now see a table of contents containing hyperlinks to results.

6 Assignment 4: Design your own component

In this assignment, we would like test whether you have learned how to work with the **FitOptiVis DSL**. Therefore, we present the following actions.

$\mathcal{A}$6.1 Based on what you have learned from the first three assignments, we kindly ask you to try to design your own component model. You are free to use examples you face in practice. Also, we encourage you to combine what has been taught in the various learning goals.

$\mathcal{A}$6.2 To visualize the changes in this assignment, save and build by repeating action *$\mathcal{A}$3.4* and action *$\mathcal{A}$3.5*.

$\mathcal{A}$6.3 After the instance has been built, the results are stored in a web-page in the **src-gen** folder that can be accessed from the project explorer. After double clicking on **src-gen**, double click the folder that starts with the name of your DSL instance and further contains the exact time it was build (i.e., year-month-day hour minute second milli-second). Finally, open **results.html**. You should now see a table of contents containing hyperlinks to results.

7 Future work

The following options and features are candidates to be added to or have already been added to the **FitOptiVis DSL**.

- Perform a model transformation on the defined system. Namely, we flatten the system in such as way that only the bottom layer components remain. The result of this transformation yields a system that we consider semantically equivalent to the original system. However, it is easier to analyze due to lacking a hierarchy.
- Add the results of the flattened system of the previous step to the overall results
- Describing inputs, outputs and budgets with quantitative values. This facilitates partial-order I/O and budget matching which in turn can be used for quality and resource management
- Provide multiple plug-ins to define a model part, which makes the **FitOptiVis DSL** customizable to meet the needs of one or few consortium partners. For instance, a component model for dataflow graphs has been developed that automatically generates a component including quantities. For another instance, the horizontal and vertical composition of components is under investigation enables multiple components to be replaced by one. Both mentioned plug-ins yield a component, which can be combined in one system model that contains a number of components

8 Survey to assess the workshop experience

We kindly request you to fill in this survey after the workshop; A bit of your time is very valuable to us. In case you want to fill in the survey at a later time, you are free to scan (or create a text file) and email it to **f.g.b.v.d.berg@tue.nl**

Name (optional, to ensure anonimity)
Institution (optional, to ensure anonimity)
Email address (optional, to ensure anonimity)

Did you know what to expect from the workshop in advance?				
totally disagree	disagree	neutral	agree	totally agree
Remarks:				

Did the workshop lead to new insights that are valuable to you regarding the Fitoptivis project?				
totally disagree	disagree	neutral	agree	totally agree
Remarks:				

Are you willing to invest in using the DSL and toolset based on what the workshop provided?				
totally disagree	disagree	neutral	agree	totally agree
Remarks:				

General remarks